Project: Auto-scaling Data Access and Business Logic

Due: 04/25/2026 12:59 AM
Extended Deadline: 04/29/2026 10:53 AM

Introduction
- Objectives
- Context & Prerequisites
- Evaluation Method
- Handout

Single Instance - MySQL
- Task Overview
- Task

Separate Reader + Writer for MySQL
- Task Overview
- Task

Horizontal Autoscaling
- Task Overview
- Task

Wrap Up
- Conclusion

Horizontal Autoscaling (Task)

Intro

Your system now has a persistent, replicated database layer and a fully operational LLM inference service. The final piece is making it elastic — able to grow and shrink automatically in response to real traffic. In this task, you will configure Kubernetes probes to make your LLM pods lifecycle-aware, so the cluster knows exactly when a pod is ready to serve requests and when it isn't. You will then attach a Horizontal Pod Autoscaler that watches CPU utilization and spins up new pods as load increases. To stress-test the system, you will run a Locust load test directly against your LLM service and watch the HPA react in real time. By the end, you will have a system that scales itself — the hallmark of a truly cloud-native architecture.

Assumptions

This task assumes that you have:

1. Completed the primer on Horizontal Pod Autoscaling, which has been added to this week's schedule.
2. Accepted the GitHub Classroom invite included in the primer. Note that GitHub Classroom may incorrectly tell you that you no longer have access to the repo after accepting — if this happens, check your email for a follow-up invite from GitHub Classroom. If the issue persists, reach out to us on the discussion platform and we'll get it sorted.

Getting to Horizontal Autoscaling

With the deployments you made from last project, and the deployments you made in the last task, you might have noticed that we did not care that much about pod creation or deletion.

To be more precise, you likely followed the following process when working with Kubernetes:

1. Apply the deployment manifest.
2. Wait for your container to start.
3. Ping the service and wait for your web service to start before doing anything more.

While this was fine when we had a finite number of pods, if we want to scale up and down our application dynamically, we want Kubernetes to know when a pod is ready to take new requests and when it clearly is not ready to take new requests.

The best-practice way of doing this is through defining probes:

> **Startup probes** are simple tests Kubernetes can run to check that our application has started and is ready to accept requests.

Readiness probes are simple tests Kubernetes can run to check if our application is ready to accept another request, or if its busy processing data.

Liveness probes are tests Kubernetes can run to check if a pod is failing, and needs to be restarted. For this stateless service, we will not need this.

For CPU-intensive applications, enabling both startup and readiness probes is usually a best-practice, as they allow us to be mindful about our resource utilization and ensure that Kubernetes is aware of how the service is doing before scaling the application accordingly.

Your first step will be to add both a `readinessProbe` and a `startupProbe` to your LLM **k8s/llm/deployment.yaml** file, calling the `healthcheck` endpoint to ensure that no traffic is lost.

Read the docs page, and add the appropriate section to your manifest.

https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/

If you want an example configuration set - see the following:

For the startup probe, ensure that you can fail the initial check at most 5 times, and that the waiting period between failures is 5 seconds.

For the readiness probe, ensure that you can fail the check at most once before re-routing requests, and that the waiting period between probes is 3 seconds.

Full disclosure - this isn't the best set of parameters, more a good set of defaults for this program:

We know that, during startup, loading the LLM onto memory is going to be somewhat expensive. It does seem ridiculous for it to take more than 5 seconds, so if it takes around 25 seconds on our hardware, it seems reasonable to say that there is an error here.

We also know that our compute is CPU-bound, and will take some time (around 30+ seconds per inference). We just want to make sure our cluster is always seeing more traffic, and increase traffic when possible.

Horizontal Autoscaling with Kubernetes HPA

Now that your system is ready to scale with load, add a `HorizontalPodAutoscaler` for this task.

Go through the official documentation on the Horizontal Pod Autoscaling before proceeding. Link here:
https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/

Information

If you need to define a HorizontalPodAutoscaler in the future, know you should **always** set CPU and memory limits before doing so. This will ensure your pods are stable and will not error out if you scale too high or too low.

In order for a Horizontal Autoscaler to know when to scale up or down your service, you will need metrics from the Kubernetes metrics server (for example the `metrics-server` add-on on AKS, or an equivalent installation on your cluster).

Your task is to create an HPA configuration that:

Starts off with a single replica.

Scales to the maximum number of replicas possible for your set of nodes.
Utilizes CPU utilization to scale up and down, so that the system will dynamically scale when there are enough requests.
The `scaleTargetRef.name` field must match the `metadata.name` of your deployment in `deployment.yaml`.
Store this configuration in the **k8s/llm/hpa.yaml** file
Reference the **hpa.yaml** configuration in the `kustomization` file

Grader Specific Resource Namings

The grader expects that the deployment name for your LLM is exactly **llmapp-deployment**. Inspect the `k8s/llm/deployment.yaml` to ensure that the deployment name is exactly **llmapp-deployment**. When in doubt, you can run `kubectl get deployment` to see whether the name matches what we're looking for.
Similarly, the service name for your LLM service should be exactly **llmapp-service**. When in doubt, you can run `kubectl get svc` to see whether the name matches what we're looking for.

> **CPU resource requests are required.** The HPA calculates utilization as `actual usage / requested amount`, so without a `resources.requests.cpu` value defined in your deployment's container spec, the HPA will be unable to compute metrics and will show `cpu: <unknown>`. Make sure your LLM deployment specifies a CPU request before applying the HPA.

Disable Istio SideCar Injection

Expose your LLM service as a LoadBalancer resource before running the scaling test. This allows you to send requests directly to the service, without routing through authentication or Istio.

In your LLM `deployment.yaml`, disable Istio sidecar injection by adding the `sidecar.istio.io/inject: "false"` annotation to the pod spec. By default, Istio injects an Envoy sidecar proxy into every pod in the namespace, which intercepts all traffic and enforces mesh policies like mTLS and authorization rules. Disabling it ensures that LoadBalancer requests reach your service directly, without being blocked by the mesh.

```
spec:
  template:
    metadata:
      annotations:
        sidecar.istio.io/inject: "false"
```

> **Preventing pod crashes under load.** The `llama-cpp-python` library is not thread-safe, meaning concurrent requests to the same pod will corrupt the model's internal tensor state and cause a segfault (exit code 139). To prevent this, add an `asyncio.Lock()` to your FastAPI app so that only one inference request is processed at a time per pod. Additionally, set `max_tokens=256` instead of `None` in your inference call — leaving it as `None` allows the model to generate beyond its context window boundary, which also triggers a crash under load.

Commit and Push your Changes

After Flux has reconciled, you should see the hpa in your cluster:

```
kubectl get hpa

NAME                REFERENCE                       TARGETS
llmapp-deployment   Deployment/my-app-deployment    cpu: <s
```

If your HPA and LLM `deployment.yaml` are configured correctly, the HPA should display actual CPU metrics from your pod. If you see `cpu: <unknown>`, verify that:

- The `scaleTargetRef.name` in your HPA matches the `metadata.name` of your deployment exactly.
- Your deployment container spec defines a `resources.requests.cpu` value.

Validate

Query your LLM Service Endpoint

Run the curl command below and make sure you can get a successful respond from your LLM service before proceeding:

```
curl http://LLM_SERVICE_LOAD_BALANCER_IP/api?message=hi
```

Simple Load Test

You should have a **loadtest** folder in your project root directory. We used this to test our LLM service in the project on **Serverless and Microservice Patterns**. If you don't have this, please refer to the instructions in the Knative task to see how to set it up. Update the `timeout` in the **locustfile** to `300`.

In order to test that your system is scalable and stable, we expect that you will be able to handle requests when you run the following loadtest command with minimal failures (our load test will run around 100 queries on your system - we expect around 90 queries to pass. In doing so, you've managed to get your RPS around three times higher compared to a single node!)

```
locust --headless -H http://LLM_SERVICE_LOAD_BALANCER_IP -u
```

Visualize your HPA and Pod

While your LoadTest is running, you should check to see how your HPA is responding to the load the pods are getting:

```
azureuser@qavm-new-sub:~/cloud-native-handout-mikesambou$ k

NAME          REFERENCE                       TARGETS
my-app-hpa    Deployment/llmapp-deployment    cpu: 213%/50%

azureuser@qavm-new-sub:~/cloud-native-handout-mikesambou$ k

NAME                                  READY   STATUS      RE
llmapp-deployment-7fdfcdbd74-74h22    1/1     Running     0
llmapp-deployment-7fdfcdbd74-98lvt    0/1     Pending     0
```

When you feel confident on your scaling policy, run the submitter from the **loadtest** directory:

Hints

1. You can configure both CPU and Memory resources in the pod
2. Ensure that your loadtest runs successfully. Don't attempt to make a submission if your loadtest is timing out
3. Monitor the HPA to read the resource utilization -- kubectl get hpa --watch
4. Check the status of your LLM pods to make sure they're not crashing

Install Locust

The submitter assumes that you've installed locust globally. If you haven't please do so.
If you're using a virtual environment to run your locust test, please activate it in the terminal where your submission will be made.

Submission should be made from your repository root directory. The submitter assumes that your locustfile is located in the loadtest directory.

```
./submitter -t task3
```

Warning

In order to test your code, we run a 15 minute load test. This will take some time. We suggest thoroughly testing your policy before running the submitter, as this test can be a little hard in our experience.

← Previous Page Next Page →

Report an Issue? © 2026 Copyright Sail() Platform